

Validation and Performance Analysis of a 16 by 16 INT8 Systolic Array

Signed INT8 inputs and INT32 accumulation on the 16 by 16 systolic array

The validation campaign passed all tests: 112 test points, 3,360 measured runs, and 336 warmups, with zero recorded mismatches. The large sweep extended validated square dimensions from 256 through 1024. Useful wall throughput increased across the sweep and approached a plateau; no throughput collapse was observed in this tested range.

The current sequential Linux HPS driver remains the main performance constraint. At 1024 cubed, median driver wall time was 15.254 seconds and useful throughput was 0.1408 GOPS. Completion waits account for approximately 60% of wall time. These results establish correctness and scaling for this implementation; they do not establish an FPGA speedup over an optimized CPU library.

Suite	Points	Measured runs	Warmups	Result
Original full suite	108	3240	324	PASS
Large square sweep	4	120	12	PASS
Combined	112	3360	336	PASS

Large square results

Dimension	Median wall s	p95 wall s	Maximum s	Useful GOPS
256	0.2904	0.2906	0.2906	0.1155
512	2.0463	2.0467	2.0470	0.1312
768	6.5926	6.5930	6.5932	0.1374
1024	15.2542	15.2580	15.2646	0.1408

All large squares have 100% geometric padding efficiency. Doubling 256 to 512 increases useful work eightfold, while median latency rises about 7.05 times. From 768 to 1024, throughput improves only about 2.4%, which is consistent with diminishing gains as fixed output-tile overhead is amortized.

Architecture and implementation phases

Phase 1 Tiling and Block Accumulation CFG_ACCUMULATE

The 4 by 4 baseline resets accumulators for each independent operation. The 16 by 16 implementation retains partial sums across K tiles using CFG_ACCUMULATE and captures output after the final step. Matrix decomposition in the HPS driver extends execution beyond a single array tile. Correctness across multiple K steps and square sizes through 1024 validates this execution path.

Phase 2 Partial Tiles and Runtime Dimensions ACT_M ACT_K ACT_N

Active dimension registers configure runtime matrix dimensions. The measured HPS path handles edge tiles by zero padding to 16 by 16 and returning only valid output elements. Irregular shapes and dimensions around tile boundaries validate this padded path. Native rectangular execution using the active dimensions has a known feed timing limitation; the reported results do not validate that path.

Phase 3 Double Buffering Ping Pong Matrix Registers

Ping pong operand banks extend the baseline's single bank organization and allow the next tile to load while another bank supplies computation. This mechanism is present in the RTL. The sequential HPS submission schedule does not measure its potential transfer overlap benefit.

Phase 4 Pipelining and Concurrent Load Compute Drain

The updated RTL provides concurrent operand loading, computation, and result draining in place of the baseline's sequential stages. The present driver waits for each tile step before reusing its transfer buffers. The timing breakdown therefore characterizes sequential execution rather than the maximum throughput of the concurrent pipeline.

Test method and correctness coverage

The board tests use the ARM Cortex A55 Linux HPS, a 16 by 16 signed INT8 array, and INT32 results. Register access is through the HPS MMIO bridge; tile payloads are transferred using the two MSGDMA engines and the reserved DDR buffers at 0xb8000000 and 0xb8010000. The measurements characterize Linux HPS execution of the 16 by 16 implementation. The 4 by 4 baseline was evaluated separately through the JTAG Master Bridge.

Group	Points	Coverage
Repeat	3	64, 128, and 256 cubed
Correctness	36	Six shapes and six signed patterns
Boundary	36	15/16/17, 31/32/33, 63/64/65; each axis and squares
Scaling and shape	33	K sweep, output dimensions, square, tall, wide and irregular
Large	4	256, 512, 768 and 1024 cubed

Each point has three verified warmups followed by 30 verified measured runs. Random inputs use deterministic recorded seeds. Correctness patterns cover full-range signed random inputs, negative-only inputs, all -128, all 127, alternating extremes, and zero A with random B. Irregular shapes exercise padding and accumulation. Large cases use signed random inputs; the extreme patterns were tested in the original suite through K = 256.

Expected results are calculated using a scalar CPU implementation of matrix multiplication. Every output element is compared after each invocation. All 112 test points passed, with 30 verified measured runs per point and no mismatches. Statistics are calculated from the individual measured runs, excluding warmups.

Timing and metric definitions

FPGA wall time uses steady_clock around run_tiled_gemm. It includes packing, reset and buffer initialization, MMIO and DMA submission, completion waits, and output decoding. Allocation, input generation, CPU reference calculation, verification, console output and CSV writes are outside that interval. Suites use busy polling with poll_us = 0 and no tile logging.

Useful operations count one multiply and one addition as two operations: $2 \times M \times K \times N$. Useful GOPS divide this count by median wall time in seconds and by one billion. Tile steps are $\text{ceil}(M/16) \times \text{ceil}(K/16) \times \text{ceil}(N/16)$; output tiles are $\text{ceil}(M/16) \times \text{ceil}(N/16)$. Padding efficiency is useful operations divided by tile steps $\times 8192$. It describes geometry, not measured PE utilization.

Median is the mean of the middle pair for 30 observations. Nearest-rank p95 is the 29th sorted value. Warmups are excluded. Stage medians are independent and need not sum to median wall time. Hardware cycle counts cover the existing PE reset, compute and flush interval; any cycle-to-time conversion assumes the configured 100 MHz clock.

Square scaling through 1024

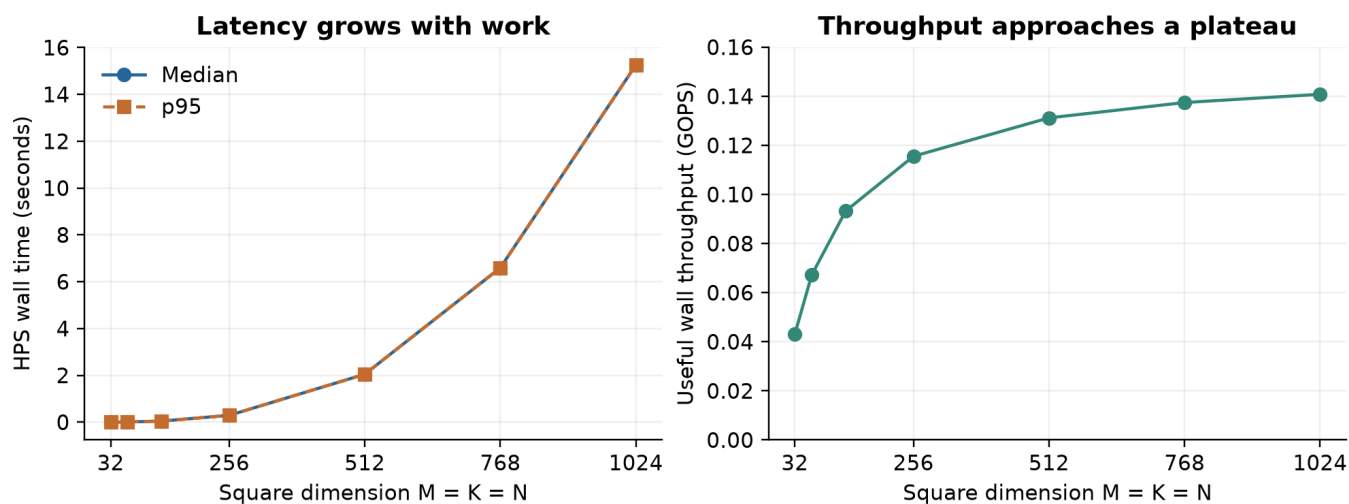


Figure 1 Measured square scaling. Sizes 32–128 use the original scaling group; sizes 256–1024 use the large suite. Each point summarizes 30 verified measured runs. p95 and median nearly overlap on this scale.

Dimension	Tile steps	Output tiles	Median wall ms	Useful GOPS
32	8	4	1.52	0.0431
64	64	16	7.80	0.0672
128	512	64	44.99	0.0932
256	4,096	256	290.45	0.1155
512	32,768	1,024	2,046.35	0.1312
768	110,592	2,304	6,592.61	0.1374
1024	262,144	4,096	15,254.19	0.1408

Median useful throughput rises from 0.0431 GOPS at 32 cubed to 0.1408 GOPS at 1024 cubed. The rise slows with size: the 512-to-768 gain is about 4.8%, followed by about 2.4% from 768 to 1024. Increasing latency is expected as useful work grows; it is not itself evidence of a scalability failure.

The repeated 256 cubed point is consistent across sessions: the original scaling group measured 290.572 ms and the large suite measured 290.448 ms, a difference of about 0.043%. This supports joining the curves at 256, while preserving the two runs as separate observations.

Where the driver spends time

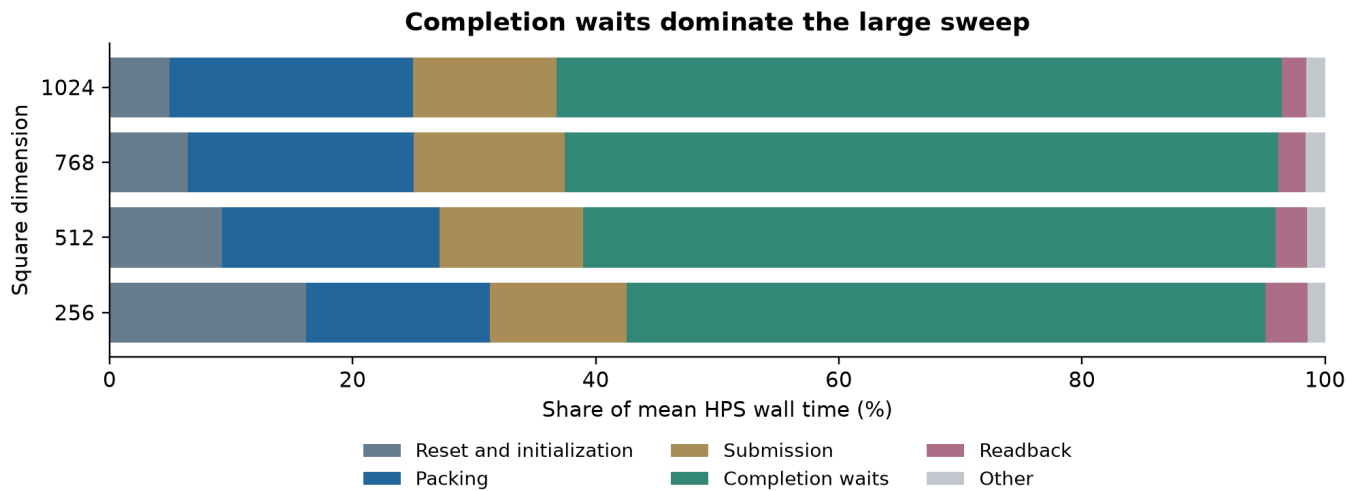


Figure 2 Additive breakdown of mean wall time from the raw measured runs. Each fraction is mean stage time divided by mean wall time; other time is the residual. This avoids stacking independent stage medians.

Stage	256 median ms	1024 median ms
Reset and initialization	47.06	755.59
Packing	43.87	3,054.96
Submission	32.59	1,795.33
Completion waits	152.71	9,104.06
Readback	10.11	308.56
Other	4.09	235.37

At 1024 cubed, completion waits contribute about 59.7% of wall time and packing about 20.0%. Submission contributes about 11.8%; reset and initialization about 5.0%. The wait interval includes accelerator execution, DMA movement, and software polling. This breakdown cannot attribute the full wait time to any one of those components.

Reset cost is incurred per output tile, while packing, submission, and completion repeat per K tile step. Their differing growth rates explain why larger K can amortize output-tile overhead, even though every additional step still incurs transfer and host costs. These measurements characterize sequential tile submission. Concurrent transfers and ping pong buffer utilization are not exercised by this execution schedule.

At 1024 cubed, the recorded median hardware count is 16,523,264 cycles, or 165.233 ms at 100 MHz. That compute/reset/flush interval is much shorter than the 15.254-second wall time, but omits transfer, capture and host costs. Dividing useful work by that interval must not be presented as end-to-end throughput.

Tile boundaries and padding

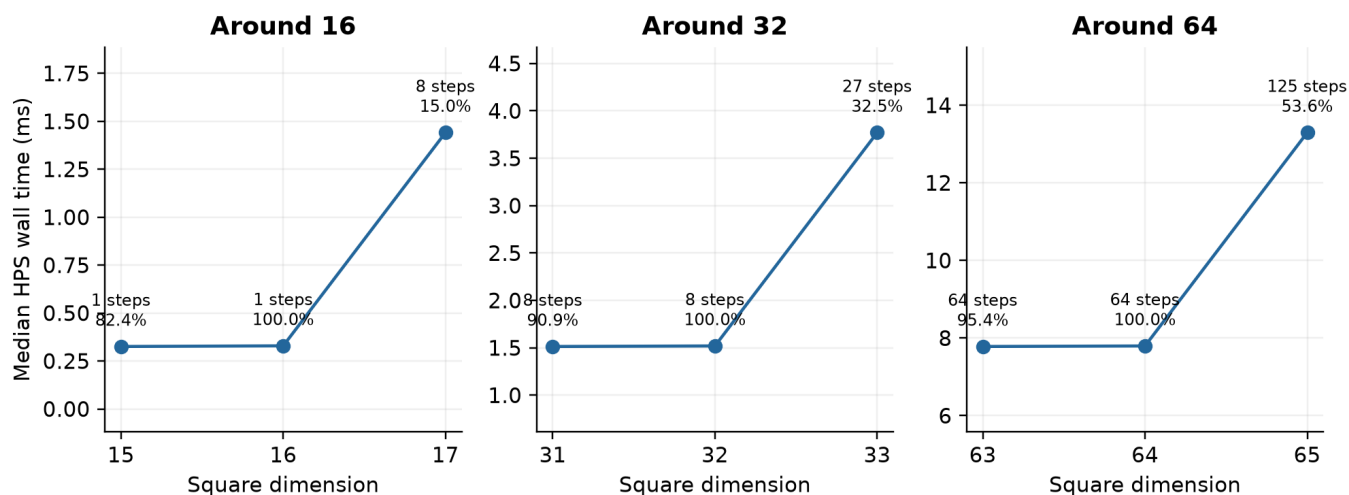


Figure 3 Square dimensions immediately around tile boundaries. Annotations show tile steps and geometric padding efficiency. Crossing a boundary adds work for padded tiles.

Boundary	Steps before to after	Latency increase	Useful GOPS change
16 to 17	1 to 8	337.9%	0.0249 to 0.0068
32 to 33	8 to 27	148.1%	0.0431 to 0.0191
64 to 65	64 to 125	70.5%	0.0673 to 0.0413

At 64 cubed the array executes 64 tile steps with no padding. At 65 cubed it executes 125 steps, with only 53.6% geometric padding efficiency. Median latency rises from 7.795 to 13.295 ms, approximately 70.5%, although useful operations increase only about 4.8%. Useful throughput falls from 0.0673 to 0.0413 GOPS. This is a measured boundary penalty, not evidence that all matrices larger than 64 perform poorly.

The original suite also varies M, K, and N separately around each boundary while holding the other axes at 64. The square plot highlights the combined effect when all three axes cross a boundary together.

Accumulation and matrix shape

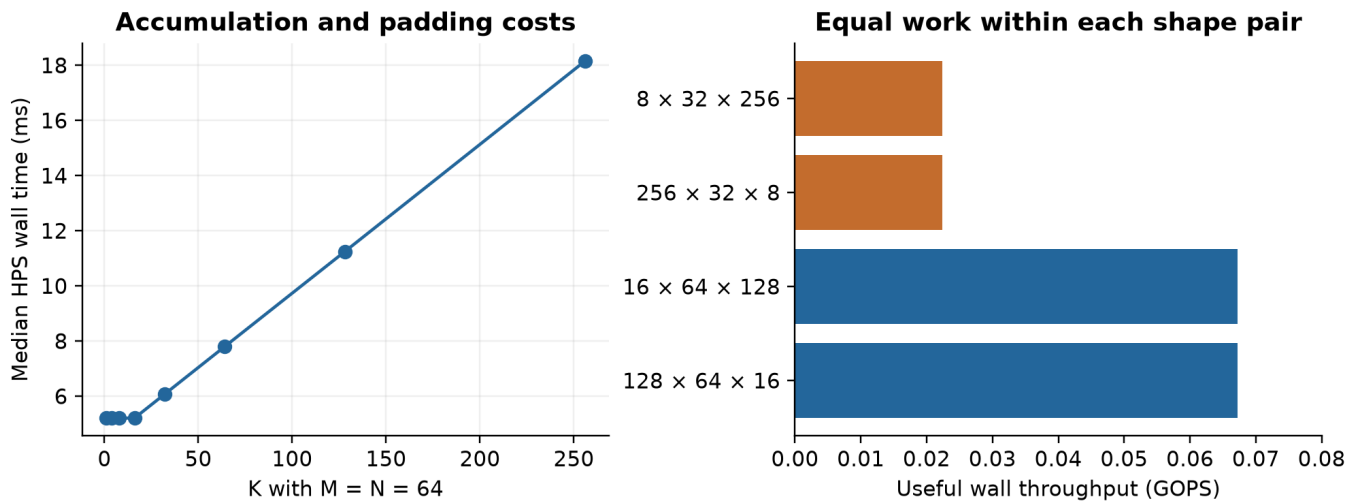


Figure 4 K scaling with $M = N = 64$ and shape comparisons. Labels use $M \times K \times N$. The aligned pair performs 262,144 useful operations per shape; the thin pair performs 131,072. Work matches within each pair, not across both pairs.

K values 1, 4, 8, and 16 each use one padded K step per output tile and have nearly the same median latency, about 5.21 ms. Useful throughput nevertheless rises from 0.00157 to 0.02518 GOPS because the fraction of useful work rises. From K = 16 through 256, latency increases with accumulation steps, while useful throughput reaches 0.11549 GOPS at K = 256.

Changing M or N at fixed K = 64 produces almost symmetric results. For aligned output dimensions 16 through 256, useful throughput remains close to 0.0672 GOPS while latency grows with output-tile count. This is sustained overhead rather than a sharp throughput cliff.

Shape M K N	Tile steps	Padding efficiency	Median ms	Useful GOPS
128 × 64 × 16	32	100%	3.900	0.06721
16 × 64 × 128	32	100%	3.900	0.06722
256 × 32 × 8	32	50%	5.849	0.02241
8 × 32 × 256	32	50%	5.849	0.02241

Tall and wide shapes have essentially equal latency within each pair. Across pairs, the thin shapes perform half as many useful operations, but still execute 32 padded tile steps. They have 50% padding efficiency, twice as many output tiles, and lower K. Their latency is about 5.85 ms rather than 3.90 ms. This combines padding and per-output-tile costs; it is not a comparison at equal useful work across pairs.

Repeatability and CPU comparison

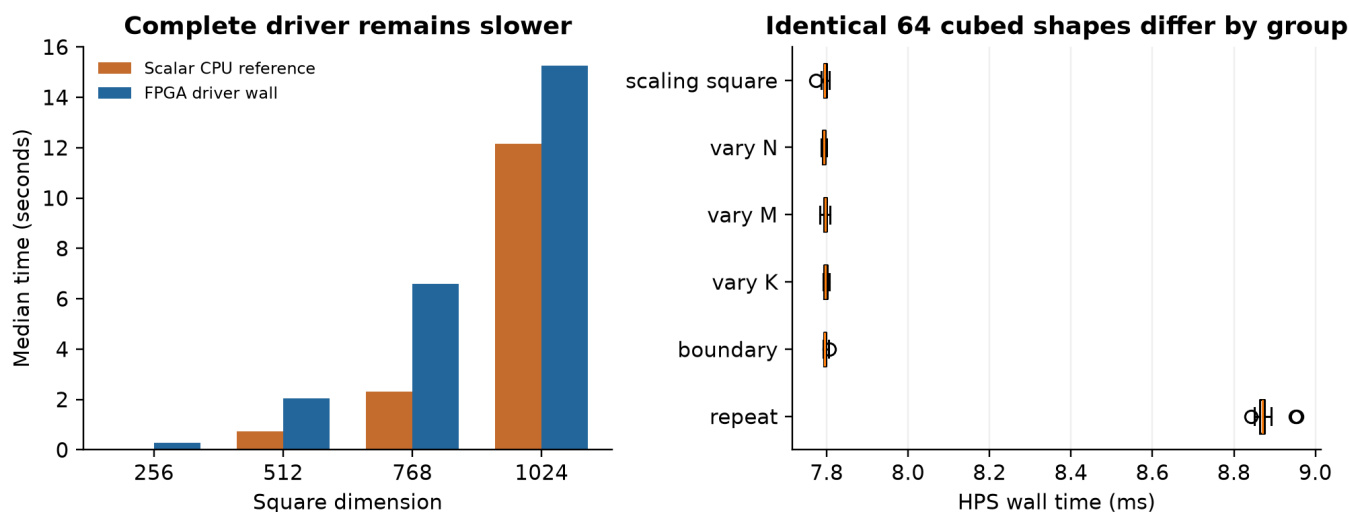


Figure 5 Left: independent median CPU reference and FPGA driver times in the large suite. Right: distributions of 30 measured runs for identical 64 cubed shapes in original suite order; boxes show quartiles and the line shows the median.

Dimension	CPU median s	FPGA median s	CPU time divided by FPGA time
256	0.0267	0.2904	0.092×
512	0.7229	2.0463	0.353×
768	2.3230	6.5926	0.352×
1024	12.1505	15.2542	0.797×

The FPGA driver is slower than this scalar reference at every large size. At 1024 cubed the CPU median is 12.150 seconds versus 15.254 seconds for the driver. The reference is a simple scalar loop, not an optimized BLAS or NEON implementation. Its memory access behavior changes with size, so the narrowing gap alone does not prove an accelerator architectural improvement.

Within the large sweep, p95 is less than 0.051% above median for every size. This describes these 30 observations; it does not establish rare-event tail latency or stability across repeated sessions.

Across original suite groups, identical 64 cubed cases differ: the initial repeat group has an 8.870 ms median, whereas later groups are about 7.79–7.80 ms. Its readback median is 1.325 ms versus about 0.486 ms in the later scaling group, with a smaller reset difference as well. The cause of this timing variation remains undetermined. Repeating the same workload at the beginning and end of multiple sessions would distinguish consistent execution behavior from effects associated with test order.

Conclusions

This test campaign demonstrates correct signed INT8 GEMM with INT32 accumulation, edge padding, multiple output tiles, and K accumulation across 112 test points. The large sweep validates random square matrices through 1024. Useful wall throughput rises toward approximately 0.14 GOPS, while padding boundaries cause localized drops. No general throughput collapse was observed.

The principal performance limitation is the gap between the counted array interval and complete driver execution. Completion waits and packing dominate the large cases. The sequential execution schedule does not establish the throughput of the concurrent RTL pipeline. The CPU comparison uses a scalar reference, and no power or energy measurements were collected.

Execution procedure

Build `gemm_test.cpp` with `gemm_suite.hpp` and `gemm_metrics.hpp` alongside it using `aarch64-none-linux-gnu-g++ -O3 -Wall -Wextra -std=c++17 gemm_test.cpp -o gemm_test`. After programming the existing combined image, boot with `run sa_boot` so the DMA DDR region is reserved. Mount the SD FAT partition at `/mnt` and run each suite with a new output prefix.

```
sudo ./gemm_test --suite all --output-prefix /mnt/full_suite_02
```

```
sudo ./gemm_test --suite large --output-prefix /mnt/large_suite_02
```

```
sync
```